AUTOMATION SKILLS PORTFOLIO · CIVIL / TRANSPORTATION

ADAPTIVE URBAN TRAFFIC MANAGEMENT SYSTEM

with Vehicle Loop Sensor Optimization



TECHNICAL PROJECT MANUAL · REV. A

By Sipho Lucky Sibanda

Platform: Siemens S7-1500 (SCL) / CODESYS-portable Structured Text

Simulation & Portfolio Engineering Build · Not for Traffic Cabinet Deployment

0i

FRONT MATTER

Document Control & Disclaimer

This document is a self-authored technical project manual produced as part of a personal engineering portfolio. It describes the design, control philosophy, and simulated validation of an adaptive, loop-actuated urban traffic signal controller, built to demonstrate actuated phase sequencing, fairness-aware green allocation, pedestrian safety logic, and emergency vehicle preemption for automation, controls, and civil/transportation engineering roles.

The system described here was developed and tested in simulation only (PLCSIM-style forcing of inputs and desktop review). No part of this project has been installed, commissioned, or verified on a physical traffic cabinet or live road, and it must not be treated as a certified traffic signal controller.

HONEST LIMITATION — Portfolio project, not a certified signal controller

A real traffic signal controller is paired with an independent hardware conflict monitor, yellow and all-red intervals calculated per ITE guidance from actual approach speed and geometry, and formal approval from the responsible road authority. Figures, thresholds, and I/O in this manual are engineering-realistic but illustrative.

Document Title	Adaptive Urban Traffic Management System — Technical Manual
Author	Sipho Lucky Sibanda
Revision	A
Document Type	Portfolio Technical Manual (Simulation)
Target Platform	Siemens S7-1500 (TIA Portal / SCL) — CODESYS-portable
Related Repository	traffic-adaptive-utms
Series	Automation Skills Portfolio — Civil Engineering Technology

ii

FRONT MATTER

Contents

01	Industry Context: Actuated vs. Fixed-Time Signals
02	System Architecture & Intersection Overview
03	Hardware & Instrumentation Specification
04	I/O List
05	Control Philosophy: Gap-Out, Fairness & Preemption
06	PLC Logic Walkthrough
07	HMI Design & the Live Intersection Map
08	Alarm Philosophy & Fail-Safe Design
09	Testing, Commissioning & FAT Procedures
10	Limitations, Real-World Deltas & Future Work
A	Appendix A — I/O Quick Reference
B	Appendix B — Full Structured Text Listing
C	Appendix C — Glossary
—	About the Author

iii

FRONT MATTER

Executive Summary

This is the sixth deliberate discipline shift in the wider automation portfolio, moving from process and power systems toward real-time public infrastructure - a field where the people most affected by a control decision are not operators at all, but every driver and pedestrian passing through the intersection. A fixed-time signal gives every approach the same green regardless of whether anyone is actually there, wasting cycle time on an empty street while the opposite approach queues.

FB_Traffic_Intersection_Control implements a genuinely actuated controller: in-pavement loop detectors extend green only while vehicles keep arriving (a vehicle extension gap), a green ends early when that gap expires with no further demand (gap-out), and a hard maximum green ceiling prevents any single stream from holding the intersection indefinitely. A longest-wait register per direction drives a fairness override, cutting a green short - once its own minimums are met - when the opposite approach has waited too long.

Two safety behaviours sit underneath all of that and cannot be overridden by efficiency logic: a pedestrian clearance interval that no vehicle-side decision may shortcut, and emergency vehicle preemption that transitions safely through all-red rather than snapping abruptly, then recovers from a clean all-red state rather than resuming mid-phase.

What follows documents the traffic engineering context, architecture, hardware assumptions, full I/O list, control philosophy, complete annotated code, HMI design, alarm philosophy, and the functional test procedure used to validate the logic in simulation.

01

ADAPTIVE TRAFFIC CONTROL

Industry Context: Actuated vs. Fixed-Time Signals

Traffic signals fall broadly into two families. Fixed-time (pre-timed) controllers run the same cycle regardless of conditions - simple, predictable, and well-suited to a coordinated corridor with very regular traffic patterns. Actuated controllers read detectors and adapt in real time, which is dramatically more efficient at an isolated intersection with variable or unpredictable demand.

The vocabulary of actuated control

- Minimum green — the shortest green any phase may show, once started, regardless of demand. It exists so a driver is never given a green that vanishes before they can reasonably react to it.
- Vehicle extension (gap) — each new detector actuation restarts a short timer; as long as vehicles keep arriving within that gap, the green is extended.
- Gap-out — when the gap timer expires with no new actuation, the stream has effectively dried up and the green may terminate.
- Maximum green — a hard ceiling that ends the green even if actuations never stop, protecting the cross street from indefinite delay.

NOTE — Why these terms matter beyond this project

These are the same terms a NEMA-style traffic cabinet controller's programming interface uses directly. Implementing them faithfully - rather than inventing a bespoke "smart traffic" vocabulary - is deliberate: it makes this project's logic legible to anyone who has configured a real signal controller.

Pedestrians and emergency vehicles are not edge cases

A real intersection must accommodate pedestrians crossing at their own, much slower speed, and emergency vehicles that legitimately need right-of-way immediately. Both are routinely treated as exceptions bolted onto a vehicle-focused design, and both are, in this project, first-class parts of the logic rather than afterthoughts.

02

ADAPTIVE TRAFFIC CONTROL

System Architecture & Intersection Overview

Four approaches (North, South, East, West) each carry an in-pavement loop detector and an emergency-vehicle preemption receiver. Two signal head groups - North-South and East-West - are driven by a single PLC function block, alongside pedestrian call buttons for each crossing direction.

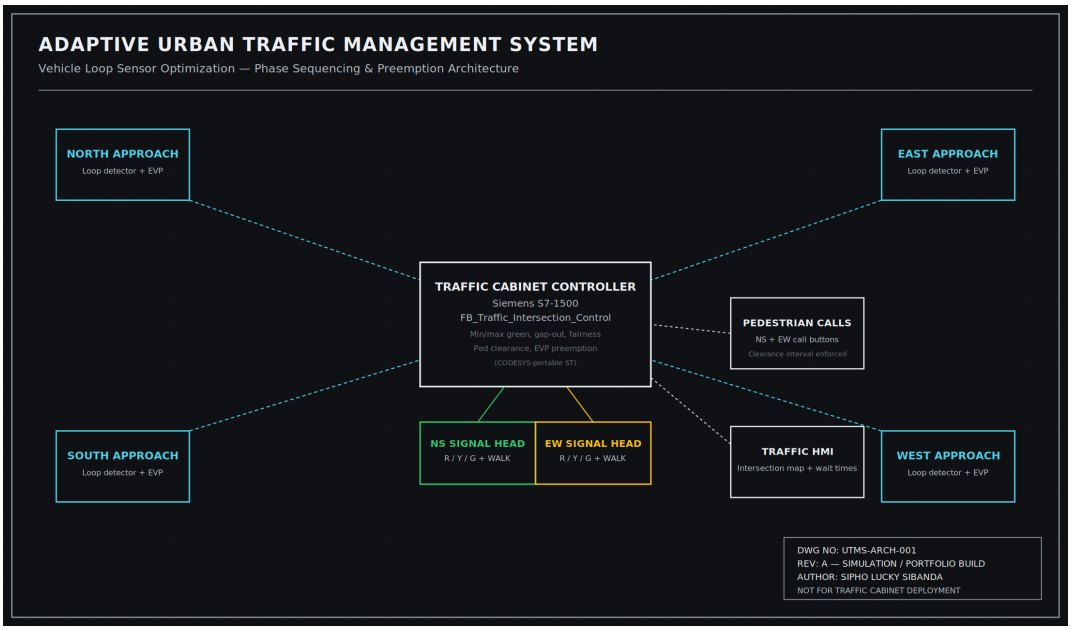


Figure 2.1 — Control architecture. Dashed lines represent PLC input signal and configuration paths; solid lines represent the controller's signal head outputs.

Control hierarchy

- Field layer — four loop detectors, four EVP receivers, two pedestrian call buttons, two signal head groups.
- Control layer — a single PLC function block, FB_Traffic_Intersection_Control, hosted on a Siemens S7-1500 (or CODESYS-based traffic cabinet controller), running phase sequencing, fairness, pedestrian safety, and preemption.
- Supervisory layer — a traffic management HMI displaying the live intersection state and each direction's accumulated wait.

Why two phases (NS/EW), not four or eight

A real intersection with protected left turns typically runs a NEMA dual-ring, multi-phase structure. This project deliberately models just two phases so the actuated-control, fairness, and preemption logic - where the real engineering interest lies - stays legible rather than being buried under phase-count bookkeeping. Chapter 10 names this plainly as a scope boundary.

03

ADAPTIVE TRAFFIC CONTROL

Hardware & Instrumentation Specification

As with the rest of this portfolio, the logic here is platform-portable but written against a concrete reference platform so timings are grounded rather than arbitrary.

Component	Representative Spec	Role
Traffic Cabinet Controller	Siemens SIMATIC S7-1500	Executes FB_Traffic_Intersection_Control
Inductive Loop Detector	In-pavement wire loop + detector card, per approach	Vehicle presence/actuation
EVP Receiver	Optical or GPS-based preemption receiver, per approach	Emergency vehicle request
Pedestrian Push Button	ADA-compliant, per crossing direction	Pedestrian demand + WALK request
Signal Heads	LED R/Y/G + pedestrian WALK indication	Right-of-way indication
Conflict Monitor	Independent hardware safety device (not modelled here)	Forces flash on conflicting-green fault

DESIGN NOTE — Yellow and all-red intervals are calculated, not chosen

This project's 3-second yellow and 2-second all-red are illustrative defaults. On a real installation both are calculated per ITE guidance from the intersection's approach speed and geometry - a faster approach needs a longer yellow, and a wider intersection needs a longer all-red for the last vehicle to clear. Getting these wrong is a genuine safety defect, not a tuning preference.

04

ADAPTIVE TRAFFIC CONTROL

I/O List

The table below is the working I/O list for FB_Traffic_Intersection_Control (see also docs/IO_List.md in the repository, and Appendix A of this manual).

Inputs

Tag	Description	Signal
DI_Loop_N / _S / _E / _W	In-pavement vehicle presence loop, per approach	Digital x4
DI_PedCall_NS / DI_PedCall_EW	Pedestrian call buttons	Digital x2
DI_EVP_N / _S / _E / _W	Emergency vehicle preemption requests	Digital x4
DI_System_Enable	Master enable	Digital

Outputs

Tag	Description	Signal
DO_NS_Green / _Yellow / _Red	North-South signal head	Digital x3
DO_EW_Green / _Yellow / _Red	East-West signal head	Digital x3
DO_NS_WalkSignal / DO_EW_WalkSignal	Pedestrian WALK indications	Digital x2
CurrentPhase	Current signal phase (enumeration)	Internal / HMI tag
WaitTime_NS_s / WaitTime_EW_s	Longest-wait registers per direction	REAL x2
Alarm_PreemptionActive / SystemStatus	Alarm + status text	Digital / STRING

05

ADAPTIVE TRAFFIC CONTROL

Control Philosophy: Gap-Out, Fairness & Preemption

Four ideas define this controller's behaviour, layered so each one can only restrict what the one before it permits - never override a safety minimum.

5.1 Actuated green: three independent ways to end it

Once a green has satisfied both its minimum green and its pedestrian clearance interval, three separate conditions can end it: reaching maximum green (the hard ceiling), gapping out with no demand left on that approach, or the fairness override (5.2). They are evaluated in that order of specificity, and none of them can fire before the non-negotiable minimums have been met.

5.2 Longest-wait fairness: why a gap-out controller still needs it

A plain actuated controller has a subtle weakness: a steady stream of vehicles on one street can keep resetting the gap timer, holding that green until maximum green expires - and maximum green is typically long. Meanwhile, the cross street's drivers wait the entire time. The longest-wait register closes that gap: it accumulates only while a direction has genuine demand and is not currently green, and once it crosses a threshold, it ends the current green early - after minimums are met - rather than letting the stream run to its ceiling.

DESIGN NOTE — The wait register is not a free-running clock

It only accumulates while that direction has real demand and is not currently green. A direction with no vehicles or pedestrians waiting accumulates nothing, so an idle street never builds up a phantom wait that could trigger a needless fairness override against a busy one.

5.3 Pedestrian clearance is a floor, not a preference

Once a WALK indication has been given, the pedestrian clearance timer must complete before the phase may end - regardless of gap-out, max green, or fairness. A pedestrian already in the crosswalk is not part of the vehicle detection picture, and the logic treats their safety as a hard constraint rather than a variable to optimise against.

5.4 Emergency preemption: fast, but never abrupt

When an emergency vehicle request arrives, the controller doesn't snap instantly to green for the requesting direction - that would trap vehicles mid-intersection exactly as the cross traffic is released. It forces a safe all-red transition first, then holds green for the requesting direction until the request clears, then recovers through a clean all-red rather than resuming mid-phase. It also waits for an active pedestrian clearance interval to finish before beginning that transition.

06

ADAPTIVE TRAFFIC CONTROL

PLC Logic Walkthrough

This chapter walks through FB_Traffic_Intersection_Control section by section. The full listing is reproduced in Appendix B.

6.1 Demand detection and the longest-wait registers

```
DemandNS := DI_Loop_N OR DI_Loop_S OR DI_PedCall_NS;
DemandEW := DI_Loop_E OR DI_Loop_W OR DI_PedCall_EW;

T_WaitNS(IN := DemandNS AND (CurrentPhase <> PH_NS_GREEN), PT := MaxWait_PT);
T_WaitEW(IN := DemandEW AND (CurrentPhase <> PH_EW_GREEN), PT := MaxWait_PT);
WaitTime_NS_s := TIME_TO_REAL(T_WaitNS.ET) / 1000.0;
WaitTime_EW_s := TIME_TO_REAL(T_WaitEW.ET) / 1000.0;
```

Listing 6.1 — The literal "which lane has waited longest" counters.

6.2 Ending a green: three reasons, evaluated only after minimums

```
IF T_MinGreen.Q AND T_PedClearance.Q THEN
    IF T_MaxGreen.Q THEN
        CurrentPhase := PH_NS_YELLOW; // Ceiling reached
    ELSIF T_Gap.Q AND (NOT DemandNS) THEN
        CurrentPhase := PH_NS_YELLOW; // Gapped out, no demand left
    ELSIF T_Gap.Q AND DemandEW AND (WaitTime_EW_s >= TIME_TO_REAL(MaxWait_PT)/1000.0) THEN
        CurrentPhase := PH_NS_YELLOW; // Fairness override
    END_IF
END_IF
```

Listing 6.2 — Both minimums (vehicle AND pedestrian) gate everything below them.

6.3 Resting instead of cycling through an empty phase

```
PH_ALLRED_1:
    T_Clearance(IN := TRUE, PT := AllRedTime_PT);
    IF T_Clearance.Q THEN
        T_Clearance(IN := FALSE);
        IF DemandEW THEN
            CurrentPhase := PH_EW_GREEN;
        ELSIF DemandNS THEN
            CurrentPhase := PH_NS_GREEN; // Nobody waiting across - rest in green
        END_IF
        // If neither has demand, remains in PH_ALLRED_1 until a call arrives.
    END_IF
```

Listing 6.3 — No wasted cycle through a phase nobody is waiting on.

6.4 Emergency preemption, with the pedestrian guard

```
IF Alarm_PreemptionActive AND (CurrentPhase <> PH_PREEMPT_TRANSITION)
   AND (CurrentPhase <> PH_PREEMPT_HOLD) THEN
    PreemptDirection_IsNS := EVP_Requested_NS;
    IF PedActiveThisPhase THEN
        ; // Never cut a pedestrian clearance interval short, even for an emergency vehicle
    ELSE
        ReturnPhaseAfterPreempt := CurrentPhase;
        CurrentPhase := PH_PREEMPT_TRANSITION;
        T_Clearance(IN := TRUE, PT := YellowTime_PT + AllRedTime_PT);
    END_IF
END_IF
```

Listing 6.4 — The transition begins only once any active pedestrian clearance has completed.

07

ADAPTIVE TRAFFIC CONTROL

HMI Design & the Live Intersection Map

The HMI mockup (hmi/index.html in the repository) is a top-down view of the intersection itself: roads, crosswalks, animated cars, in-pavement loop pads that light up when a car sits on them, and signal heads reflecting the live phase - matching what a traffic management centre operator would expect to see for a single monitored intersection.

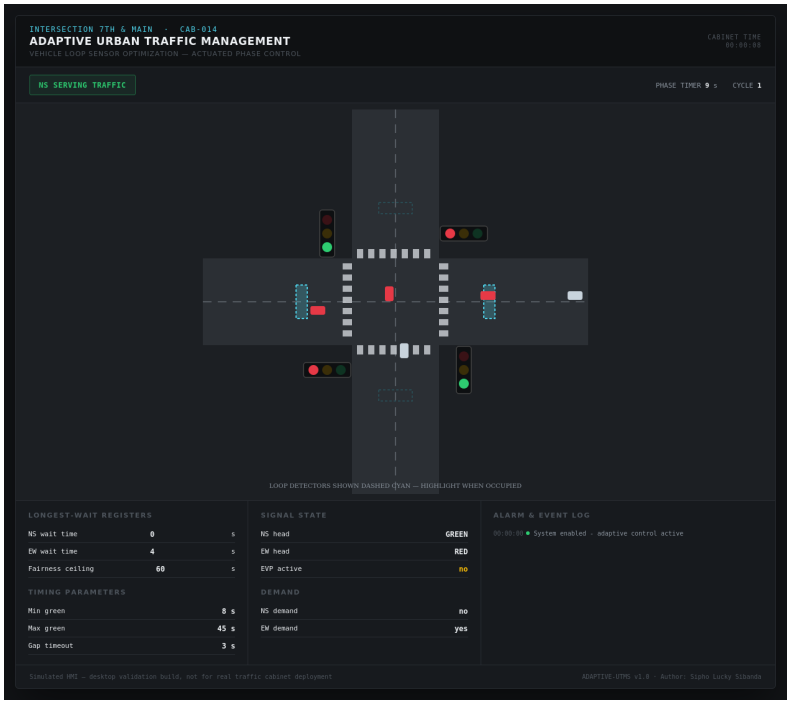


Figure 7.1 — NS serving traffic: the NS signal heads show green, EW shows red with an EW car waiting on its loop pad (lit cyan) and its wait register accumulating in the side panel.

Design decisions

- Amber on dark asphalt — a seventh distinct visual identity in this portfolio, deliberately built around real traffic-signal red/amber/green on a neutral grey-black rather than another tinted "console" palette, since the signal colours themselves are the subject matter.
- The loop pads are shown, not hidden — dashed cyan outlines that fill when occupied, making the actuation mechanism visible rather than an invisible input behind the signal heads.
- Cars actually stop and go — each approach's cars queue at the stop line on red and proceed on green, so the loop pad occupancy and the demand registers are produced by the simulation rather than faked independently.
- An emergency preemption event is scripted into the demo — so the all-red transition and hold-green recovery from Chapter 5.4 is something a viewer sees happen, not just a line in the documentation.

08

ADAPTIVE TRAFFIC CONTROL

Alarm Philosophy & Fail-Safe Design

This controller carries a single alarm deliberately - a signal controller's real safety envelope is enforced by what it structurally refuses to do, not by a long list of annunciated conditions an operator might miss.

Condition	Meaning	Expected Response
Alarm_PreemptionActive	An emergency vehicle has requested right-of-way	Informational - system self-manages; confirm it clears once the vehicle has passed

Fail-safe defaults, summarised

- System disabled → the sequencer forces PH_ALLRED_1 immediately, rather than holding whichever green was last active.
- Minimum green and pedestrian clearance are checked before any early-termination reason, so no efficiency feature can shorten a green below its safety floor.
- Preemption transitions through all-red and recovers through all-red - it never switches directly from one green to the other.

09

ADAPTIVE TRAFFIC CONTROL

Testing, Commissioning & FAT Procedures

The function block was validated against twelve functional test cases, anchored by one test specifically proving the fairness override fires inside a green's normal operating range. The full procedure is in docs/Testing_Procedures.md; the matrix is reproduced below.

#	Test Case	Expected Result
1	Minimum green enforced	Green holds until MinGreen_PT regardless of demand loss
2-3	Gap-out and vehicle extension	Gap expires with no actuation; a new actuation resets it
4	Maximum green is a hard ceiling	Green ends at the ceiling despite continuous demand
5	Rest instead of cycling an empty phase	Returns to the active side rather than serving an empty one
6	Longest-wait fairness override	A busy green ends early once the other side's wait crosses the ceiling
7	Pedestrian clearance cannot be cut short	Phase holds until the clearance timer completes
8-9	EVP interrupts; still respects ped clearance	Transitions via all-red; waits out an active pedestrian interval
10	Preemption recovery	Returns via all-red, not directly to a green
11	Simultaneous EVP requests	NS takes priority per the defined order
12	System disable	Forces a safe all-red

ENGINEERING NOTE — Why Test 6 is the one that demonstrates "intelligence"

Tests 1 through 4 describe a conventional actuated signal - genuinely useful, but not distinct from any standard traffic cabinet controller. Test 6 is where the longest-wait registers do something a plain gap-out/max-green controller wouldn't: cut a green short, inside its normal operating range, specifically because the other direction's accumulated wait crossed a fairness threshold.

10

ADAPTIVE TRAFFIC CONTROL

Limitations, Real-World Deltas & Future Work

Naming the gap between a strong simulation and a certifiable traffic signal controller is part of the engineering, consistent with every other project in this portfolio.

What a real installation would add

- An independent hardware conflict monitor — a separate safety device that forces a flashing state on any conflicting-green fault, entirely independent of this software.
- A full NEMA dual-ring, multi-phase structure — protected left turns, overlaps, and concurrent phase timing this two-phase scope deliberately omits (Chapter 2).
- Calculated yellow and all-red intervals — derived per ITE guidance from actual approach speed and geometry rather than fixed illustrative defaults.
- Detector fault handling — a stuck-on or dead loop is a real, common failure mode; this project assumes its detectors report truthfully.

10.1 Hazard & safeguard register (HAZOP-style)

Hazard	Cause	Safeguard in This Design
Conflicting greens shown simultaneously	A logic or output fault	Named gap - requires an independent hardware conflict monitor
Pedestrian struck during a cut-short phase	An efficiency feature shortening green below clearance	Pedestrian clearance gates every early-termination reason
Cross-street starvation	A steady stream resetting the gap timer indefinitely	Longest-wait fairness override plus a hard maximum green
Stuck detector holding a green forever	A failed loop reporting permanent occupancy	Named gap - maximum green bounds the damage, but detector fault detection is not modelled

Where this project could go next

- Extend to a full dual-ring structure with protected left-turn phases.
- Add detector fault detection (a loop that never clears, or never actuates).
- Coordinate two adjacent intersections with a shared offset, rather than treating each as an isolated controller.

0A

APPENDIX

Appendix A — I/O Quick Reference

Tag	Dir.	Type	Notes
DI_Loop_N / _S / _E / _W	IN	BOOL x4	Per-approach loop
DI_PedCall_NS / DI_PedCall_EW	IN	BOOL x2	Pedestrian calls
DI_EVP_N / _S / _E / _W	IN	BOOL x4	Preemption requests
DI_System_Enable	IN	BOOL	Master enable
DO_NS_Green / Yellow / Red	OUT	BOOL x3	NS signal head
DO_EW_Green / Yellow / Red	OUT	BOOL x3	EW signal head
DO_NS_WalkSignal / DO_EW_WalkSignal	OUT	BOOL x2	Pedestrian WALK
CurrentPhase	OUT	ENUM	8-state sequencer
WaitTime_NS_s / WaitTime_EW_s	OUT	REAL x2	Longest-wait registers
Alarm_PreemptionActive	OUT	BOOL	EVP in progress
SystemStatus	OUT	STRING	HMI display

APPENDIX

0B

Appendix B — Full Structured Text Listing

Complete, unedited listing of src/Traffic_Intersection_Control.st, read directly from the repository source so this appendix cannot drift from the code.

```
(*
================================================================================
  PROJECT : Adaptive Urban Traffic Management System (UTMS)
            Vehicle Loop Sensor Optimization
  MODULE : FB_Traffic_Intersection_Control
  PLATFORM : IEC 61131-3 Structured Text
            Written against Siemens SCL (TIA Portal, S7-1500) syntax,
            portable to CODESYS-based traffic cabinet controllers.
  AUTHOR : Sipho Lucky Sibanda
  CONTEXT : A fixed-time traffic signal gives every direction the same
            green duration regardless of whether anyone is actually
            there - wasting cycle time on an empty street while the
            opposite approach queues up. This function block runs a
            genuinely actuated intersection: in-pavement loop
            detectors extend green only while vehicles keep arriving,
            a longest-wait register prevents any approach from being
            starved, and an emergency vehicle preemption path can
            safely interrupt normal operation within one scan.

  ENGINEERING BASIS (simulated against, for realism only):
    - NEMA-style actuated signal control concepts: minimum green,
      maximum green, vehicle extension ("gap"), and gap-out - the
      standard vocabulary real traffic cabinet controllers use.
    - Pedestrian clearance interval: once a WALK indication has been
      given, the signal may not leave green (or extend yellow/all-red)
      until the minimum time needed for a pedestrian already in the
      crosswalk to finish crossing has elapsed.
    - Emergency Vehicle Preemption (EVP): a detector (e.g. an Opticom-
      style optical/GPS system) on an approach requests immediate,
      safe right-of-way for an approaching emergency vehicle, cutting
      short the normal cycle through a safe (not abrupt) transition.
================================================================================
*)

TYPE E_SignalPhase :
(
    PH_NS_GREEN,
    PH_NS_YELLOW,
    PH_ALLRED_1,
    PH_EW_GREEN,
    PH_EW_YELLOW,
    PH_ALLRED_2,
    PH_PREEMPT_TRANSITION,
    PH_PREEMPT_HOLD
```

Listing B.1 — FB_Traffic_Intersection_Control source (part 1 of 8).

```
);
END_TYPE

FUNCTION_BLOCK FB_Traffic_Intersection_Control
VAR_INPUT
    DI_Loop_N : BOOL; // Vehicle present, North approach
    DI_Loop_S : BOOL; // Vehicle present, South approach
    DI_Loop_E : BOOL; // Vehicle present, East approach
    DI_Loop_W : BOOL; // Vehicle present, West approach
    DI_PedCall_NS : BOOL; // Pedestrian call button, N-S crosswalk
    DI_PedCall_EW : BOOL; // Pedestrian call button, E-W crosswalk
    DI_EVP_N : BOOL; // Emergency vehicle preemption request, per approach
    DI_EVP_S : BOOL;
    DI_EVP_E : BOOL;
    DI_EVP_W : BOOL;
    DI_System_Enable : BOOL;
END_VAR

VAR_OUTPUT
    DO_NS_Green : BOOL := FALSE;
    DO_NS_Yellow : BOOL := FALSE;
    DO_NS_Red : BOOL := TRUE;
    DO_EW_Green : BOOL := FALSE;
    DO_EW_Yellow : BOOL := FALSE;
    DO_EW_Red : BOOL := TRUE;
    DO_NS_WalkSignal : BOOL := FALSE;
    DO_EW_WalkSignal : BOOL := FALSE;
    CurrentPhase : E_SignalPhase := PH_ALLRED_1;
    WaitTime_NS_s : REAL := 0.0; // "Which lane has waited longest" registers
    WaitTime_EW_s : REAL := 0.0;
    Alarm_PreemptionActive : BOOL := FALSE;
    SystemStatus : STRING[28] := 'STANDBY';
END_VAR

VAR
    MinGreen_PT : TIME := T#8S;
    MaxGreen_PT : TIME := T#45S;
    GapTimeout_PT : TIME := T#3S; // Vehicle extension / "gap" interval
    YellowTime_PT : TIME := T#3S;
    AllRedTime_PT : TIME := T#2S;
    PedClearance_PT : TIME := T#12S;
    MaxWait_PT : TIME := T#60S; // Fairness override ceiling
```

Listing B.1 — FB_Traffic_Intersection_Control source (part 2 of 8).

```
    T_MinGreen : TON;
    T_MaxGreen : TON;
    T_Gap : TON;
    T_Clearance : TON; // Shared: yellow / all-red / preempt transition
    T_PedClearance : TON;
    T_WaitNS : TON;
    T_WaitEW : TON;

    DemandNS : BOOL;
    DemandEW : BOOL;
    PedActiveThisPhase : BOOL;
    EVP_Requested_NS : BOOL;
    EVP_Requested_EW : BOOL;
    PreemptDirection_IsNS : BOOL;
    ReturnPhaseAfterPreempt : E_SignalPhase;

    Prev_Loop_N : BOOL; Prev_Loop_S : BOOL;
    Prev_Loop_E : BOOL; Prev_Loop_W : BOOL;
    AnyNewArrival_NS : BOOL;
    AnyNewArrival_EW : BOOL;
END_VAR

// ------------------------------------------------------------------
// 1. DEMAND DETECTION
// ------------------------------------------------------------------
DemandNS := DI_Loop_N OR DI_Loop_S OR DI_PedCall_NS;
DemandEW := DI_Loop_E OR DI_Loop_W OR DI_PedCall_EW;

AnyNewArrival_NS := (DI_Loop_N AND NOT Prev_Loop_N) OR (DI_Loop_S AND NOT Prev_Loop_S);
AnyNewArrival_EW := (DI_Loop_E AND NOT Prev_Loop_E) OR (DI_Loop_W AND NOT Prev_Loop_W);
Prev_Loop_N := DI_Loop_N; Prev_Loop_S := DI_Loop_S;
Prev_Loop_E := DI_Loop_E; Prev_Loop_W := DI_Loop_W;

// ------------------------------------------------------------------
// 2. LONGEST-WAIT REGISTERS
// Accumulate only while that direction has genuine demand and is
// NOT the one currently green - the literal "which lane has been
// waiting the longest" counter.
// ------------------------------------------------------------------
T_WaitNS(IN := DemandNS AND (CurrentPhase <> PH_NS_GREEN), PT := MaxWait_PT);
T_WaitEW(IN := DemandEW AND (CurrentPhase <> PH_EW_GREEN), PT := MaxWait_PT);
WaitTime_NS_s := TIME_TO_REAL(T_WaitNS.ET) / 1000.0;
WaitTime_EW_s := TIME_TO_REAL(T_WaitEW.ET) / 1000.0;
```

Listing B.1 — FB_Traffic_Intersection_Control source (part 3 of 8).

```
// ------------------------------------------------------------------
// 3. EMERGENCY VEHICLE PREEMPTION - evaluated before the normal
// sequencer, and able to interrupt it from any state.
// ------------------------------------------------------------------
EVP_Requested_NS := DI_EVP_N OR DI_EVP_S;
EVP_Requested_EW := DI_EVP_E OR DI_EVP_W;
Alarm_PreemptionActive := EVP_Requested_NS OR EVP_Requested_EW;

IF Alarm_PreemptionActive AND (CurrentPhase <> PH_PREEMPT_TRANSITION) AND (CurrentPhase <>
PH_PREEMPT_HOLD) THEN
    PreemptDirection_IsNS := EVP_Requested_NS; // NS takes priority if both request simultaneously
    IF PedActiveThisPhase THEN
        // Never cut a pedestrian clearance interval short, even for
        // an emergency vehicle - the transition waits for it.
        ; // T_PedClearance continues running below; transition begins once it completes
    ELSE
        ReturnPhaseAfterPreempt := CurrentPhase;
        CurrentPhase := PH_PREEMPT_TRANSITION;
        T_Clearance(IN := TRUE, PT := YellowTime_PT + AllRedTime_PT);
    END_IF
END_IF

// ------------------------------------------------------------------
// 4. MAIN PHASE STATE MACHINE
// ------------------------------------------------------------------
IF NOT DI_System_Enable THEN
    CurrentPhase := PH_ALLRED_1;
END_IF

CASE CurrentPhase OF

    PH_NS_GREEN:
        DO_NS_Green := TRUE; DO_NS_Red := FALSE; DO_EW_Red := TRUE;
        DO_NS_WalkSignal := TRUE; PedActiveThisPhase := TRUE;
        T_MinGreen(IN := TRUE, PT := MinGreen_PT);
        T_MaxGreen(IN := TRUE, PT := MaxGreen_PT);
        T_Gap(IN := NOT AnyNewArrival_NS, PT := GapTimeout_PT);
        IF AnyNewArrival_NS THEN T_Gap(IN := FALSE); END_IF
        T_PedClearance(IN := TRUE, PT := PedClearance_PT);
```

Listing B.1 — FB_Traffic_Intersection_Control source (part 4 of 8).

```
        IF T_MinGreen.Q AND T_PedClearance.Q THEN
            // Three independent reasons a green can end, once the
            // non-negotiable minimums (vehicle AND pedestrian) are met:
            IF T_MaxGreen.Q THEN
                CurrentPhase := PH_NS_YELLOW; // Ceiling reached
            ELSIF T_Gap.Q AND (NOT DemandNS) THEN
                CurrentPhase := PH_NS_YELLOW; // Gapped out, no demand left
            ELSIF T_Gap.Q AND DemandEW AND (WaitTime_EW_s >= TIME_TO_REAL(MaxWait_PT)/1000.0) THEN
                CurrentPhase := PH_NS_YELLOW; // Fairness override
            END_IF
        END_IF
        IF CurrentPhase <> PH_NS_GREEN THEN
            DO_NS_WalkSignal := FALSE; PedActiveThisPhase := FALSE;
            T_MinGreen(IN := FALSE); T_MaxGreen(IN := FALSE); T_Gap(IN := FALSE); T_PedClearance(IN
:= FALSE);
            T_Clearance(IN := TRUE, PT := YellowTime_PT);
        END_IF

    PH_NS_YELLOW:
        DO_NS_Green := FALSE; DO_NS_Yellow := TRUE;
        T_Clearance(IN := TRUE, PT := YellowTime_PT);
        IF T_Clearance.Q THEN
            DO_NS_Yellow := FALSE; DO_NS_Red := TRUE;
            T_Clearance(IN := TRUE, PT := AllRedTime_PT);
            CurrentPhase := PH_ALLRED_1;
        END_IF

    PH_ALLRED_1:
        T_Clearance(IN := TRUE, PT := AllRedTime_PT);
        IF T_Clearance.Q THEN
            T_Clearance(IN := FALSE);
            IF DemandEW THEN
                CurrentPhase := PH_EW_GREEN;
            ELSIF DemandNS THEN
                CurrentPhase := PH_NS_GREEN; // Nobody waiting across - rest in green, don't waste a
cycle
            END_IF
            // If neither has demand, remains in PH_ALLRED_1 (resting at red both ways) until a call
arrives.
        END_IF
```

Listing B.1 — FB_Traffic_Intersection_Control source (part 5 of 8).

```
    PH_EW_GREEN:
        DO_EW_Green := TRUE; DO_EW_Red := FALSE; DO_NS_Red := TRUE;
        DO_EW_WalkSignal := TRUE; PedActiveThisPhase := TRUE;
        T_MinGreen(IN := TRUE, PT := MinGreen_PT);
        T_MaxGreen(IN := TRUE, PT := MaxGreen_PT);
        T_Gap(IN := NOT AnyNewArrival_EW, PT := GapTimeout_PT);
        IF AnyNewArrival_EW THEN T_Gap(IN := FALSE); END_IF
        T_PedClearance(IN := TRUE, PT := PedClearance_PT);

        IF T_MinGreen.Q AND T_PedClearance.Q THEN
            IF T_MaxGreen.Q THEN
                CurrentPhase := PH_EW_YELLOW;
            ELSIF T_Gap.Q AND (NOT DemandEW) THEN
                CurrentPhase := PH_EW_YELLOW;
            ELSIF T_Gap.Q AND DemandNS AND (WaitTime_NS_s >= TIME_TO_REAL(MaxWait_PT)/1000.0) THEN
                CurrentPhase := PH_EW_YELLOW;
            END_IF
        END_IF
        IF CurrentPhase <> PH_EW_GREEN THEN
            DO_EW_WalkSignal := FALSE; PedActiveThisPhase := FALSE;
            T_MinGreen(IN := FALSE); T_MaxGreen(IN := FALSE); T_Gap(IN := FALSE); T_PedClearance(IN
:= FALSE);
            T_Clearance(IN := TRUE, PT := YellowTime_PT);
        END_IF

    PH_EW_YELLOW:
        DO_EW_Green := FALSE; DO_EW_Yellow := TRUE;
        T_Clearance(IN := TRUE, PT := YellowTime_PT);
        IF T_Clearance.Q THEN
            DO_EW_Yellow := FALSE; DO_EW_Red := TRUE;
            T_Clearance(IN := TRUE, PT := AllRedTime_PT);
            CurrentPhase := PH_ALLRED_2;
        END_IF

    PH_ALLRED_2:
        T_Clearance(IN := TRUE, PT := AllRedTime_PT);
        IF T_Clearance.Q THEN
            T_Clearance(IN := FALSE);
            IF DemandNS THEN
                CurrentPhase := PH_NS_GREEN;
            ELSIF DemandEW THEN
                CurrentPhase := PH_EW_GREEN;
            ELSE
                CurrentPhase := PH_ALLRED_1; // Rest at red both ways until a call arrives
            END_IF
```

Listing B.1 — FB_Traffic_Intersection_Control source (part 6 of 8).

```
        END_IF

    PH_PREEMPT_TRANSITION:
        // Safe transition from whatever was running: force red on both
        // until the shared clearance timer (yellow + all-red worth of
        // time) has elapsed, then hold green for the requesting direction.
        DO_NS_Green := FALSE; DO_NS_Yellow := FALSE; DO_NS_Red := TRUE;
        DO_EW_Green := FALSE; DO_EW_Yellow := FALSE; DO_EW_Red := TRUE;
        DO_NS_WalkSignal := FALSE; DO_EW_WalkSignal := FALSE;
        IF T_Clearance.Q THEN
            T_Clearance(IN := FALSE);
            CurrentPhase := PH_PREEMPT_HOLD;
        END_IF

    PH_PREEMPT_HOLD:
        IF PreemptDirection_IsNS THEN
            DO_NS_Green := TRUE; DO_NS_Red := FALSE; DO_EW_Red := TRUE;
        ELSE
            DO_EW_Green := TRUE; DO_EW_Red := FALSE; DO_NS_Red := TRUE;
        END_IF
        IF NOT Alarm_PreemptionActive THEN
            // Emergency vehicle has cleared - return to normal service,
            // starting from a clean all-red rather than resuming mid-phase.
            DO_NS_Green := FALSE; DO_EW_Green := FALSE;
            T_Clearance(IN := TRUE, PT := AllRedTime_PT);
            CurrentPhase := PH_ALLRED_1;
        END_IF

    ELSE
        CurrentPhase := PH_ALLRED_1;
END_CASE

// ------------------------------------------------------------------
// 5. STATUS TEXT
// ------------------------------------------------------------------
IF NOT DI_System_Enable THEN
    SystemStatus := 'STANDBY';
ELSIF Alarm_PreemptionActive THEN
    SystemStatus := 'EMERGENCY PREEMPTION';
ELSE
    CASE CurrentPhase OF
        PH_NS_GREEN, PH_NS_YELLOW: SystemStatus := 'NS SERVING TRAFFIC';
        PH_EW_GREEN, PH_EW_YELLOW: SystemStatus := 'EW SERVING TRAFFIC';
        ELSE SystemStatus := 'CLEARANCE / ALL RED';
```

Listing B.1 — FB_Traffic_Intersection_Control source (part 7 of 8).

```
    END_CASE
END_IF

END_FUNCTION_BLOCK
```

Listing B.1 — FB_Traffic_Intersection_Control source (part 8 of 8).

0C

APPENDIX

Appendix C — Glossary

Term	Meaning
Actuated signal	A signal whose timing responds to real-time detector input
Fixed-time signal	A signal running a pre-set cycle regardless of conditions
Minimum green	The shortest green any phase may show once started
Maximum green	A hard ceiling on green duration regardless of demand
Gap / vehicle extension	A timer restarted by each vehicle actuation, extending the green
Gap-out	Ending a green when the gap timer expires with no new actuation
All-red	A short interval where every approach shows red, clearing the intersection
EVP	Emergency Vehicle Preemption
Loop detector	An in-pavement wire loop that senses a vehicle's presence by inductance change
Conflict monitor	An independent hardware device forcing a safe flash on any conflicting-green fault
NEMA / ITE	Traffic control equipment standard / transportation engineering guidance body

CLOSING

About the Author

Sipho Lucky Sibanda is an automation and controls engineer building a multi-disciplinary portfolio spanning marine systems, avionics, architectural technology, applied AI, biotechnology, electrical power systems, civil transportation, and industrial automation. This manual documents the sixth deliberate discipline shift in that portfolio, following the same documentation standard as every other project in the series: full logic, I/O documentation, a live HMI, functional test procedures, and an honest account of what separates a strong simulation from a certifiable production system.

Repository: traffic-adaptive-utms

End of document.